CAMPUS SMART-PARKING SYSTEM

Name : B.Theerthi

Course: Programming java

Introduction

The Campus Smart-Parking System is a Java-based application designed to manage vehicle parking in a university campus. The system contains multiple floors, and each floor contains a fixed number of parking slots. Students, staff and visitors can park their vehicles, including two-wheelers, cars and electric vehicles.

The main purpose of this system is to automate parking slot allocation, vehicle entry and exit, ticket generation and parking fee calculation. The system also supports electric vehicle charging fees and free parking for students and staff holding valid monthly passes.

Java object-oriented programming concepts such as classes, objects, inheritance, polymorphism, abstraction, encapsulation and interfaces are used to develop a flexible and reusable system.

TASK 1 – UNDERSTAND THE PROBLEM AND IDENTIFY THE OBJECTS

1. Problem Understanding

The university requires a smart parking management system to manage vehicles entering and leaving its parking facility. The system must automatically identify available parking slots and assign a suitable slot to each vehicle.

The system should generate a unique parking ticket and record the vehicle's entry time. When the vehicle leaves, the system calculates the parking fee and releases the occupied slot.

2. Inputs of the System

1. Vehicle registration number.
2. Vehicle type (Two-Wheeler, Car or Electric Vehicle).
3. Floor number.
4. Parking slot number.
5. Vehicle entry time.
6. Vehicle exit time.
7. User category (Student, Staff or Visitor).
8. Monthly pass validity.
9. Electricity consumed by an electric vehicle.

3. Outputs of the System

1. Allocated parking floor and slot number.
2. Unique parking ticket number.
3. Vehicle entry and exit details.
4. Total parking duration.

5. Calculated parking fee.
6. Electric vehicle charging fee.
7. Available and occupied slot information.
8. Daily parking collection.

4. Identification of Candidate Classes

Candidate Class	Responsibility
Vehicle	Stores common vehicle information
TwoWheeler	Represents two-wheeler vehicles
Car	Represents cars
ElectricCar	Represents electric vehicles
ParkingSlot	Maintains slot availability
ParkingFloor	Manages parking slots on a floor
ParkingFacility	Manages all floors and slot allocation
ParkingTicket	Stores ticket number and entry time
ParkingUser	Stores user information
MonthlyPass	Validates free parking eligibility
ParkingService	Handles entry, exit and fee calculation

5. Business Rules

1. A parking slot cannot contain two vehicles at the same time.
2. Every vehicle must have a unique registration number.
3. Every parking ticket must have a unique ticket number.
4. A vehicle can enter only if a suitable slot is available.
5. A released slot must become available for another vehicle.

6. Parking fees depend on vehicle type and parking duration.
7. Students and staff with valid monthly passes receive free parking.
8. Visitors must pay the applicable parking fee.
9. Electric vehicles must pay an additional charging fee when charging is used.
10. Invalid parking durations and invalid slot numbers must be rejected.

6. Assumptions

- The facility contains multiple floors.
- Every floor can have a different number of parking slots.
- Each slot can accommodate only one vehicle.
- Every started hour is charged as a complete hour.
- Parking rates are assumed to be ₹10 per hour for two-wheelers and ₹30 per hour for cars.
- Electric vehicle charging costs ₹12 per kWh.
- The system records entry and exit times using Java date and time classes.

7. Alternative Concept – Objects vs Parallel Arrays

In the object-oriented approach, every vehicle is represented as an object containing registration number, vehicle type and entry time.

In the alternative approach, separate arrays are used:

- regNo[] – stores registration numbers.
- type[] – stores vehicle types.
- entryTime[] – stores entry times.

Comparison	Object-Oriented Approach	Parallel Arrays
Data organization	Data stored together in objects	Data stored separately
Searching	O(n)	O(n)
Adding a new attribute	Add a field to the class	Add another parallel array
Maintenance	Easier	More difficult
Reusability	High	Limited

Complexity Analysis:

Searching for a vehicle takes O(n) time in both approaches when a linear search is used. The object-oriented approach requires O(n) space for n vehicle objects. Parallel arrays also require O(n) total space, but adding a new attribute requires maintaining another array.

Conclusion: The object-oriented approach is selected because it provides better data organization, maintainability and extensibility.

TASK 2 – CHOOSE THE DATA REPRESENTATION

1. Data Types Used in the System

Field	Java Data Type	Reason
Slot number	int	Stores integer slot numbers
Floor number	int	Stores floor numbers
Registration number	String	Contains letters and numbers
Vehicle type	String / enum	Identifies vehicle category
Entry time	LocalDateTime	Stores date and time
Exit time	LocalDateTime	Stores date and time
Duration	long	Stores parking duration in minutes
Parking fee	double	Stores decimal fee values
Ticket number	int	Stores ticket identification number
Charging units	double	Stores electricity consumption

2. Representation of Floors and Slots

A jagged two-dimensional array is selected because different floors may contain different numbers of parking slots.

Example:

```
String[][] slots = new String[3][];


slots[0] = new String[20];
slots[1] = new String[50];
slots[2] = new String[100];
```

Here, each row represents a floor, and each column represents a parking slot.

3. Type Conversion and Casting

Parking duration is calculated in minutes and converted into billable hours.

For example:

```
int minutes = 130;
int hours = (minutes + 59) / 60;
```

Output:

3

Every started hour is charged in full.

4. Integer Division Problem

Incorrect calculation:

```
int minutes = 90;
int hours = minutes / 60;
```

Output:

1

This is incorrect because 90 minutes should be charged as 2 hours.

Correct calculation:

```
int hours = (minutes + 59) / 60;
```

Output:

2

This avoids losing the remaining minutes during integer division.

5. Input Reading

The Scanner class is used to read user input.

```
import java.util.Scanner;

public class ParkingInput {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        System.out.print("Enter vehicle number: ");
        String regNo = sc.nextLine();

        System.out.print("Enter parking duration: ");
        int minutes = sc.nextInt();

        System.out.println("Vehicle: " + regNo);
        System.out.println("Duration: " + minutes);

        sc.close();
    }
}
```

6. Alternative Concept and Complexity Analysis

Instead of a jagged array, a single-dimensional array can be used.

Index calculation:

int index = floor * slotsPerFloor + slot;

However, this representation is convenient only when each floor has the same number of slots.

An ArrayList can also store different floor arrays:

ArrayList<String[]> floors = new ArrayList<>();

Representation	Advantage	Disadvantage
Jagged array	Supports different floor sizes	Requires nested loops
One-dimensional array	Simple indexing	Best suited for equal floor sizes
ArrayList	Dynamic size	Slightly more overhead

Complexity Analysis:

- Accessing a known array position: O(1).
- Searching all slots: O(S), where S is the total number of slots.
- Space required: O(S).

Conclusion: A jagged array is selected because it supports different numbers of slots on different floors.

TASK 3 – DESIGN THE CONTROL LOGIC

1. Slot Allocation Logic

The system searches each floor and each slot. When it finds a free slot, it assigns the vehicle to that slot and returns the floor and slot numbers.

If no slot is available, it returns null.

Algorithm: Allocate Parking Slot

1. Start.
2. Receive the vehicle registration number.
3. Traverse each floor.
4. Traverse each slot on that floor.
5. Check whether the slot is empty.
6. If empty, assign the vehicle number.
7. Return the floor and slot numbers.
8. If no free slot is found, return null.
9. Stop.

2. Fee Calculation Logic

1. Start.
2. Receive the parking duration and vehicle type.
3. Calculate billable hours.
4. Identify the applicable hourly rate.
5. Multiply the billable hours by the hourly rate.

6. Return the total fee.
7. Stop.

3. Pseudocode – Slot Allocation

```
BEGIN

INPUT vehicleNo

FOR each floor f
   FOR each slot s in floor f

      IF slots[f][s] IS NULL THEN
         slots[f][s] = vehicleNo

         RETURN f, s
      END IF

   END FOR
END FOR

RETURN NULL

END
```

4. Pseudocode – Fee Calculation

```
BEGIN
```

```
INPUT minutes
INPUT vehicleType

hours = (minutes + 59) / 60

IF vehicleType = "TwoWheeler" THEN
   rate = 10

ELSE IF vehicleType = "Car" THEN
   rate = 30

ELSE
   rate = 0
END IF

fee = hours * rate

RETURN fee

END
```

5. Complete the Code – ParkingFacility.java

```
public class ParkingFacility {

   private final String[][] slots;
```

```
public ParkingFacility(int[] slotsPerFloor) {

    slots = new String[slotsPerFloor.length][];

    for (int f = 0; f < slotsPerFloor.length; f++) {
        slots[f] = new String[slotsPerFloor[f]];
    }
}

public int[] allocate(String vehicleNo) {

    for (int f = 0; f < slots.length; f++) {

        for (int s = 0; s < slots[f].length; s++) {

            if (slots[f][s] == null) {

                slots[f][s] = vehicleNo;

                return new int[]{f, s};
            }
        }
    }

    return null;
}
```

```
public boolean release(int floor, int slot) {

    if (floor < 0 || floor >= slots.length) {
        return false;
    }

    if (slot < 0 || slot >= slots[floor].length) {
        return false;
    }

    if (slots[floor][slot] == null) {
        return false;
    }

    slots[floor][slot] = null;

    return true;
}

public static void main(String[] args) {

    ParkingFacility p =
        new ParkingFacility(new int[]{2, 1});

    System.out.println(
        java.util.Arrays.toString(p.allocate("TN01AB1111")));
```

```
        System.out.println(
            java.util.Arrays.toString(p.allocate("TN02CD2222")));

        System.out.println(
            java.util.Arrays.toString(p.allocate("TN03EF3333")));

        System.out.println(
            java.util.Arrays.toString(p.allocate("TN04GH4444")));

        System.out.println(
            p.release(0, 1) + " " +
            p.release(0, 1) + " " +
            p.release(5, 0));

        System.out.println(
            java.util.Arrays.toString(p.allocate("TN04GH4444")));
    }
}
```

Expected Output

```
[0, 0]
[0, 1]
[1, 0]
null
true false false
[0, 1]
```

6. Use of Control Statements

- **if-else:** Checks whether a parking slot is available and determines the parking fee.
- **for loop:** Searches through floors and slots.
- **break:** Stops a loop when the required result is found, when appropriate.
- **continue:** Skips occupied or unsuitable slots.
- **return:** Returns the allocated position or calculated fee.

7. Alternative Concept and Complexity Analysis

The alternative approach uses a stack of free slot positions and a free-slot count for every floor.

In the original approach, allocation searches through all slots.

- Allocation: O(F × S) worst case.
- Release: O(1).
- Extra space: O(1), excluding the slot array.

With a stack of free positions:

- Allocation: O(1).
- Release: O(1).
- Extra space: O(F × S) for the free-slot stacks in the worst case.

Conclusion: The original array-search method is simple and suitable for a small parking facility. A stack-based approach is more efficient when the number of parking slots becomes very large.

TASK 4 – DESIGN CLASSES, OBJECTS AND CONSTRUCTORS

1. Class Design

The system contains several classes to represent the real-world entities.

Vehicle Class: Stores registration number and common vehicle details.

ParkingTicket Class: Stores ticket number, registration number and entry time.

ParkingSlot Class: Stores slot number and availability.

ParkingFacility Class: Manages floors and parking allocation.

ParkingService Class: Performs vehicle entry, exit and fee calculation.

2. Constructors

A constructor initializes an object when it is created.

Types of constructors:

1. Default constructor.
2. Parameterized constructor.
3. Overloaded constructor.
4. Copy constructor (user-defined).

3. Example of Constructors

class ParkingTicket {

```
int ticketNo;
String regNo;

// Default constructor
ParkingTicket() {
   this(0, "UNKNOWN");
}

// Parameterized constructor
ParkingTicket(int ticketNo, String regNo) {
   this.ticketNo = ticketNo;
   this.regNo = regNo;
}

// Copy constructor
ParkingTicket(ParkingTicket other) {
   this(other.ticketNo, other.regNo);
}

void display() {
   System.out.println(ticketNo + " " + regNo);
}

public static void main(String[] args) {

   ParkingTicket t1 =
      new ParkingTicket(101, "TN01AB1111");
```

```
        ParkingTicket t2 =
            new ParkingTicket(t1);

        t1.display();
        t2.display();
    }
}
```

Output

```
101 TN01AB1111
101 TN01AB1111
```

4. Use of this Keyword

The this keyword refers to the current object.

It is used to distinguish instance variables from constructor parameters and to call another constructor in the same class.

Example:

```
this.ticketNo = ticketNo;
```

5. Constructor Chaining

Constructor chaining avoids repeating initialization code.

Example:

```
ParkingTicket() {
   this(0, "UNKNOWN");
}
```

6. Object Memory Management

Java uses stack and heap memory.

- Stack memory stores local variables and object references.
- Heap memory stores actual objects.
- The new keyword creates objects in heap memory.
- When an object is no longer reachable, it becomes eligible for garbage collection.

7. Alternative Concept and Complexity Analysis

Instead of constructors, objects can be created using setters or static factory methods.

Method	Advantage	Disadvantage
Constructor	Initializes object immediately	Requires parameters
Setter	Allows later modification	Can create incomplete objects
Static factory	Can control object creation	Requires factory methods

For n ParkingTicket objects, creation requires O(n) total time and O(n) space under all three approaches, assuming constant-time initialization per object.

Conclusion: Parameterized constructors are preferred because they initialize objects with valid values at the time of creation.

TASK 5 – APPLY STATIC MEMBERS AND THE OBJECT LIFE CYCLE

1. Static Variables

Static variables belong to the class rather than individual objects.

In the parking system, the ticket counter is static because ticket numbers must be generated uniquely across all ticket objects.

Example:

```
class TicketCounter {

   static int count = 1000;

   int ticketNo;

   TicketCounter() {
      ticketNo = ++count;
   }

   void display() {
      System.out.println("Ticket Number: " + ticketNo);
   }

   public static void main(String[] args) {

      TicketCounter t1 = new TicketCounter();
      TicketCounter t2 = new TicketCounter();
```

```
        t1.display();
        t2.display();
    }
}
```

Output

```
Ticket Number: 1001
Ticket Number: 1002
```

2. Static Methods

Static methods can be called without creating an object.

Example:

```
class ParkingUtility {

    static void showMessage() {
        System.out.println("Welcome to Smart Parking");
    }

    public static void main(String[] args) {
        ParkingUtility.showMessage();
    }
}
```

Output

Welcome to Smart Parking

3. Static Block

A static block executes once when the class is initialized.

```
static {
   System.out.println("Parking System Initialized");
}
```

It can be used to initialize class-level data.

4. Object Life Cycle

1. An object is created using new.
2. Memory is allocated in heap memory.
3. The constructor initializes the object.
4. The object is used by the program.
5. After the vehicle leaves, the ticket may no longer be needed.
6. When no references point to the ticket object, it becomes eligible for garbage collection.

5. Object Passing

Java passes object references by value. A method receives a copy of the reference and can access the same object.

Example:

```
void printTicket(ParkingTicket ticket) {
   ticket.display();
```

}

6. Alternative Concept and Complexity Analysis

If the ticket counter is an instance variable, every ParkingFacility object maintains its own counter.

For example, two parking facilities may generate the same ticket number.

A static counter can provide a shared sequence across facilities within the same running application.

Approach	Space for k Facilities
Static counter	O(1) counter storage
Instance counter	O(k) counter storage

For n tickets, ticket objects require O(n) space in either design.

Conclusion: A static ticket counter is useful when a shared ticket-number sequence is required. For a distributed multi-gate system, a centralized or persistent ID-generation mechanism would be needed to guarantee uniqueness across separate application instances.

TASK 6 – DESIGN THE INHERITANCE HIERARCHY

1. Introduction

Inheritance is an important object-oriented programming concept that allows one class to acquire the properties and methods of another class.

In the parking system, different vehicle types share common properties such as registration number and parking duration.

Therefore, a common superclass named Vehicle is created.

2. Inheritance Hierarchy

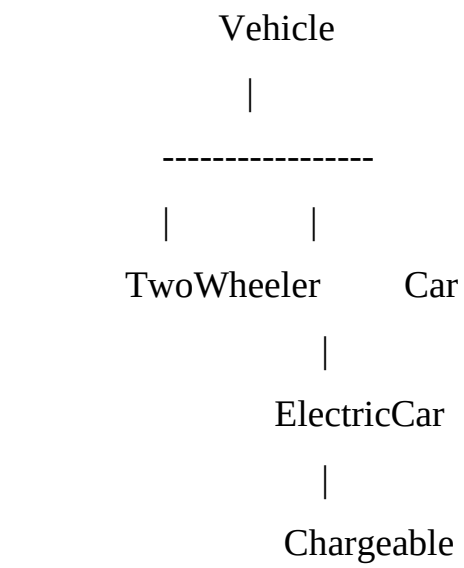
- Vehicle – Parent class.
- TwoWheeler – Child class.
- Car – Child class.
- ElectricCar – Child class of Car.

3. Types of Inheritance

The system uses:

- Hierarchical inheritance: TwoWheeler and Car inherit from Vehicle.
- Multilevel inheritance: ElectricCar inherits from Car, which inherits from Vehicle.

4. UML Class Diagram



5. Java Implementation

```
class Vehicle {

    String regNo;

    Vehicle(String regNo) {
        this.regNo = regNo;
    }

    void display() {
        System.out.println("Registration: " + regNo);
    }
}

class TwoWheeler extends Vehicle {
```

```
    TwoWheeler(String regNo) {
        super(regNo);
    }
}

class Car extends Vehicle {

    Car(String regNo) {
        super(regNo);
    }
}

class ElectricCar extends Car {

    ElectricCar(String regNo) {
        super(regNo);
    }
}

public class InheritanceDemo {

    public static void main(String[] args) {

        ElectricCar e =
            new ElectricCar("TN01AB1234");
```

```
        e.display();
    }
}
```

Output

Registration: TN01AB1234

6. Use of super Keyword

The super keyword is used to access parent-class constructors and methods.

Example:

```
ElectricCar(String regNo) {
    super(regNo);
}
```

7. Constructor Execution Order

When an ElectricCar object is created, constructors execute in the following order:

1. Vehicle constructor.
2. Car constructor.
3. ElectricCar constructor.

The parent constructor executes before the child constructor.

8. Alternative Concept – Composition

Instead of inheritance, composition can be used.

A single Vehicle class can contain a VehicleType object that stores the vehicle type and hourly rate.

Inheritance	Composition
Uses IS-A relationship	Uses HAS-A relationship
Separate subclasses	Vehicle contains VehicleType
Adding a type requires a subclass	Adding a type requires a new type configuration
Suitable for fixed categories	Flexible for changing categories

Fee calculation takes O(1) time in either approach when the rate is directly available.

Conclusion: Inheritance is selected because different vehicle categories share common properties and can have specialized behaviours.

TASK 7 – OVERRIDE AND SPECIALISE BEHAVIOUR

1. Introduction

Method overriding allows a subclass to provide its own implementation of a method inherited from the parent class.

In the parking system, different vehicles have different parking rates. Therefore, the ratePerHour() method is overridden.

2. Example

```
class Vehicle {
```

```
    double ratePerHour() {
        return 0;
    }
}

class TwoWheeler extends Vehicle {

    @Override
    double ratePerHour() {
        return 10;
    }
}

class Car extends Vehicle {

    @Override
    double ratePerHour() {
        return 30;
    }
}
```

3. Rules of Method Overriding

1. The method name must be the same.
2. The parameter list must be the same.
3. The return type must be compatible.
4. The overriding method cannot reduce the visibility of the inherited method.
5. Static methods are hidden rather than overridden.

6. The @Override annotation helps detect incorrect overriding.

4. Use of final Keyword

The final keyword can be used in three ways.

- Final variable: Its value cannot be reassigned.
- Final method: It cannot be overridden.
- Final class: It cannot be inherited.

Example:

final int MAX_SLOTS = 100;

5. Overriding Object Class Methods

Java provides methods such as toString(), equals() and hashCode().

These can be overridden to represent vehicles properly.

```
@Override
public String toString() {
   return "Vehicle: " + regNo;
}


@Override
public boolean equals(Object obj) {
   if (this == obj) return true;


   if (!(obj instanceof Vehicle)) return false;
```

```
    Vehicle other = (Vehicle) obj;

    return regNo.equals(other.regNo);
}

@Override
public int hashCode() {
    return regNo.hashCode();
}
```

Two vehicles with the same registration number are treated as equal in this example.

6. Alternative Concept – Switch Statement

Instead of overriding methods, a single method can use a switch statement.

```
double calculateRate(String type) {

    switch (type) {

        case "TwoWheeler":
            return 10;

        case "Car":
            return 30;

        default:
```

```
        return 0;
    }
}
```

Complexity Analysis

Both approaches generally calculate the fee in O(1) time for a fixed set of vehicle types.

However, when a new vehicle type is added, a switch-based design requires modifying the central calculation method. Overriding allows a new subclass to provide its own rate.

Conclusion: Method overriding provides flexibility and reduces the need for repeated conditional statements.

TASK 8 – APPLY POLYMORPHISM

1. Introduction

Polymorphism means one method or reference can represent different forms of behaviour.

Java supports compile-time polymorphism and runtime polymorphism.

2. Compile-Time Polymorphism

Method overloading is an example of compile-time polymorphism.

```
class FeeCalculator {

    double calculateFee(int minutes) {
        int hours = (minutes + 59) / 60;
        return hours * 30;
    }

    double calculateFee(int hours, double rate) {
        return hours * rate;
    }
}
```

The compiler selects the correct method based on the arguments.

3. Runtime Polymorphism

Method overriding is an example of runtime polymorphism.

A parent-class reference can store child-class objects.

Example:

```
Vehicle[] vehicles = {
    new TwoWheeler("TN01A1"),
    new Car("TN02B2"),
    new ElectricCar("TN03C3")
};
```

The JVM selects the correct overridden method at runtime.

4. Complete the Code – FeeDemo.java

```
interface Chargeable {

    double chargingFee(int kWh);
}

abstract class Vehicle {

    private final String regNo;

    protected Vehicle(String regNo) {
        this.regNo = regNo;
    }

    public String getRegNo() {
        return regNo;
```

```
    }

    protected abstract double ratePerHour();

    public double calculateFee(int minutes) {

        int hours = (minutes + 59) / 60;

        return hours * ratePerHour();
    }

    @Override
    public boolean equals(Object o) {

        return o instanceof Vehicle &&
            regNo.equals(((Vehicle) o).regNo);
    }

    @Override
    public int hashCode() {
        return regNo.hashCode();
    }
}

class TwoWheeler extends Vehicle {

    TwoWheeler(String regNo) {
```

```
        super(regNo);
    }

    @Override
    protected double ratePerHour() {
        return 10;
    }
}

class Car extends Vehicle {

    Car(String regNo) {
        super(regNo);
    }

    @Override
    protected double ratePerHour() {
        return 30;
    }
}

class ElectricCar extends Car implements Chargeable {

    ElectricCar(String regNo) {
        super(regNo);
    }
```

```
    @Override
    public double chargingFee(int kWh) {
        return kWh * 12;
    }
}

public class FeeDemo {

    public static void main(String[] args) {

        Vehicle[] parked = {
            new TwoWheeler("TN01A1"),
            new Car("TN02B2"),
            new ElectricCar("TN03C3")
        };

        for (Vehicle v : parked) {

            double fee = v.calculateFee(130);

            if (v instanceof Chargeable c) {
                fee += c.chargingFee(10);
            }

            System.out.println(
                v.getClass().getSimpleName() + " " +
                v.getRegNo() + " -> Rs " + fee
```

```
            );
        }

        System.out.println(
            new Car("TN02B2").equals(
                new ElectricCar("TN02B2")
            )
        );
    }
}
```

Expected Output

```
TwoWheeler TN01A1 -> Rs 30.0
Car TN02B2 -> Rs 90.0
ElectricCar TN03C3 -> Rs 210.0
true
```

5. Explanation of Output

- TwoWheeler: 3 hours × ₹10 = ₹30.
- Car: 3 hours × ₹30 = ₹90.
- ElectricCar: 3 hours × ₹30 + 10 kWh × ₹12 = ₹210.

The last line prints true because the equals method compares registration numbers only. Both objects have the same registration number.

In a real system, this behaviour is useful for identifying the same registered vehicle, although a registration number should also be validated to prevent duplicate active entries.

6. Alternative Concept – instanceof and Casting

Without polymorphism, the system can use multiple instanceof conditions to identify vehicle types and calculate fees.

This approach requires additional checks whenever a new vehicle type is introduced.

Polymorphism	**instanceof Checks**
Uses overridden methods	Uses conditional checks
Easy to extend	More code modifications
Supports dynamic dispatch	Requires explicit type identification
Better maintainability	Can become complicated

Both approaches have constant-time behaviour for a fixed number of types, but polymorphism offers a cleaner design.

Conclusion: Polymorphism allows the same method call to produce different results depending on the actual vehicle object.

TASK 9 – INTRODUCE ABSTRACTION

1. Introduction

Abstraction means hiding unnecessary implementation details and showing only the essential features of an object.

In the parking system, an abstract Vehicle class defines common properties and methods. Each vehicle subclass provides its own implementation of the parking rate.

2. Abstract Class

An abstract class cannot be instantiated directly.

Example:

```
abstract class Vehicle {

    String regNo;

    Vehicle(String regNo) {
        this.regNo = regNo;
    }

    abstract double calculateFee(int minutes);
}
```

The subclasses must implement the abstract method.

3. Interface

An interface defines a capability that a class can implement.

The Chargeable interface represents vehicles that support electric charging.

```
interface Chargeable {

    double chargingFee(int kWh);
}
```

4. Implementation

```
class ElectricCar extends Car implements Chargeable {

    ElectricCar(String regNo) {
        super(regNo);
    }

    public double chargingFee(int kWh) {
        return kWh * 12;
    }
}
```

5. Why Abstract Class and Interface Are Used

Abstract Class	**Interface**
Provides common state and behaviour	Defines a capability
Can have constructors	Cannot have ordinary constructors
Can contain instance variables	Fields are public, static and final

A class can extend one class A class can implement multiple interfaces

6. Multiple Interfaces

A class can implement more than one interface.

Example:

```
class ElectricCar extends Car
        implements Chargeable, Comparable<ElectricCar> {


    ElectricCar(String regNo) {
        super(regNo);
    }

    public double chargingFee(int kWh) {
        return kWh * 12;
    }

    public int compareTo(ElectricCar other) {
        return getRegNo().compareTo(other.getRegNo());
    }
}
```

7. Default and Static Methods

Interfaces can contain default and static methods.

```
interface Chargeable {
```

```
    double chargingFee(int kWh);

    default void showChargingMessage() {
        System.out.println("Charging is available");
    }

    static void showRate() {
        System.out.println("Charging rate: Rs 12 per kWh");
    }
}
```

8. Alternative Concept and Complexity Analysis

If Vehicle were an interface instead of an abstract class, common instance state and constructor initialization would need to be moved into implementing classes or another supporting class.

If Chargeable were changed into an abstract class, ElectricCar could not extend both Car and Chargeable because Java does not support multiple class inheritance.

Both abstract methods and interface methods generally have constant-time method execution, depending on the implementation.

Conclusion: An abstract class is suitable for common vehicle information, while interfaces are suitable for additional capabilities such as electric charging.

TASK 10 – ENFORCE ENCAPSULATION THROUGH PACKAGES AND ACCESS MODIFIERS

1. Introduction

Encapsulation is the process of combining data and methods into a single class and protecting the internal data from direct access.

The parking system uses private variables and public methods to protect important information.

2. Package Structure

The project can be organized into the following packages:

```
CampusSmartParking
│
├── parking.model
│   ├── Vehicle.java
│   ├── Car.java
│   ├── TwoWheeler.java
│   └── ParkingTicket.java
│
├── parking.service
│   ├── ParkingFacility.java
│   └── ParkingService.java
│
└── parking.app
    └── Main.java
```

3. Access Modifiers

Modifier	Purpose
private	Accessible only within the same class
default	Accessible within the same package
protected	Accessible within the same package and subclasses
public	Accessible from anywhere subject to normal access rules

4. Encapsulation Example

```
class ParkingSlot {

    private int slotNumber;
    private boolean occupied;

    public ParkingSlot(int slotNumber) {

        if (slotNumber <= 0) {
            throw new IllegalArgumentException(
                "Invalid slot number"
            );
        }

        this.slotNumber = slotNumber;
        this.occupied = false;
    }
```

```
    public int getSlotNumber() {
        return slotNumber;
    }

    public boolean isOccupied() {
        return occupied;
    }

    public void setOccupied(boolean occupied) {
        this.occupied = occupied;
    }
}
```

5. Validation Through Getters and Setters

The constructor validates the slot number and prevents invalid data from being stored.

Similarly, a parking duration must not be negative.

```
public void setDuration(int duration) {

    if (duration < 0) {
        throw new IllegalArgumentException(
            "Duration cannot be negative"
        );
    }
```

```
    this.duration = duration;
}
```

6. Wrapper Classes and Autoboxing

Wrapper classes convert primitive values into objects.

Examples:

- int → Integer
- double → Double
- boolean → Boolean

Autoboxing automatically converts primitive values into wrapper objects.

Example:

```
Integer slot = 10;
int number = slot;
```

This is called autoboxing and unboxing.

7. Risk of Comparing Integer Objects

Incorrect:

```
Integer a = 1000;
Integer b = 1000;


System.out.println(a == b);
```

The == operator compares object references in this situation, not numerical values.

Correct:

System.out.println(a.equals(b));

This compares the actual values.

8. Alternative Concept and Complexity Analysis

If all fields were public, external classes could directly modify important information.

For example, a negative parking duration could be assigned without validation.

Encapsulation centralizes validation in one place.

Validation of a single field takes O(1) time. For n updates, the total time is O(n).

Conclusion: Encapsulation protects the parking system from invalid data and makes the application easier to maintain.

TASK 11 – IMPLEMENT, TEST AND DEFEND THE DESIGN

1. Implementation

The complete system combines the concepts discussed in the previous tasks.

The application manages vehicle registration, parking slot allocation, ticket generation, fee calculation and vehicle exit.

2. Test Cases

Test Case	Input / Condition	Expected Result
TC01	Two-wheeler enters	Slot allocated
TC02	Car enters	Slot allocated
TC03	Electric car enters	Slot allocated
TC04	All slots occupied	No slot available
TC05	Vehicle exits	Slot released
TC06	Zero-minute duration	Zero fee
TC07	60-minute duration	One-hour fee
TC08	61-minute duration	Two-hour fee
TC09	Invalid slot number	Error or false
TC10	Duplicate active registration	Entry rejected

3. Sample Fee Calculation

Assume the following vehicles enter and leave the campus:

Vehicle	Duration	Rate	Fee
Two-Wheeler	60 minutes	₹10/hour	₹10
Car	120 minutes	₹30/hour	₹60
Visitor Car	180 minutes	₹30/hour	₹90
Electric Car	120 minutes	₹30/hour	₹60

Electric car charging consumption = 5 kWh.

Charging fee = 5 × ₹12 = ₹60.

Total collection:

₹10 + ₹60 + ₹90 + ₹60 + ₹60 = **₹280**.

If the student or staff member has a valid monthly pass, the applicable parking fee is zero.

4. Evaluation of the Design

The design follows object-oriented principles:

- Encapsulation protects vehicle and ticket data.
- Inheritance supports different vehicle types.
- Polymorphism allows different fee calculations.
- Abstraction separates common vehicle behaviour from specialized behaviour.
- Interfaces provide additional capabilities such as electric charging.

5. Limitation

The basic array-based design uses linear searching to locate vehicles and available slots. When the parking facility becomes very large, searching can take more time.

A HashMap can be used to improve vehicle lookup.

6. Alternative Concept – Array vs ArrayList vs HashMap

Structure	Searching for Vehicle	Space
Array	O(n)	O(n)
ArrayList	O(n)	O(n)
HashMap	O(1) average	O(n)

A HashMap keyed by registration number is suitable for finding active parking tickets quickly.

Conclusion: The system satisfies the main parking requirements and demonstrates Java object-oriented programming concepts. Further improvements can be made by using dynamic data structures and database storage.

TASK 12 – RE-EVALUATE AND ADAPT THE DESIGN

1. Changed Requirements

The university introduces additional requirements:

1. Add e-bikes with a flat parking fee.
2. Add delivery vans charged at double the car rate between 9 AM and 5 PM.
3. Allow visitors to purchase a one-day pass.
4. Extend the facility to five floors.
5. Allow different numbers of parking slots on each floor.

A. IMPACT ANALYSIS

1. Changes in the Original System

The original system supports two-wheelers, cars and electric cars. The updated system introduces e-bikes and delivery vans.

The pricing rules are also modified.

Original Requirement New Requirement

Two-wheelers	Existing
Cars	Existing
Electric cars	Existing
E-bikes	Newly added
Delivery vans	Newly added
Monthly pass	Existing
One-day visitor pass	Newly added
Three floors	Five floors

2. Classes Affected

- Vehicle: Extended to support additional vehicle categories.
- ParkingFacility: Modified to support five floors.
- ParkingService: Updated to handle new pricing rules.
- ParkingPass: Extended to support one-day visitor passes.
- PricingRule: Introduced to manage different pricing policies.

3. Array Design

A jagged array is suitable because each floor may contain a different number of slots.

Example:

int[] slotsPerFloor = {20, 30, 40, 50, 60};

This allows the system to represent five floors with different capacities.

B. EVALUATION OF THE ORIGINAL DESIGN

The original inheritance design can support new vehicle categories by creating additional subclasses.

However, if pricing rules are distributed across multiple classes, modifying the system whenever a new pricing condition is introduced can become difficult.

For example, delivery van pricing depends on both vehicle type and time of day.

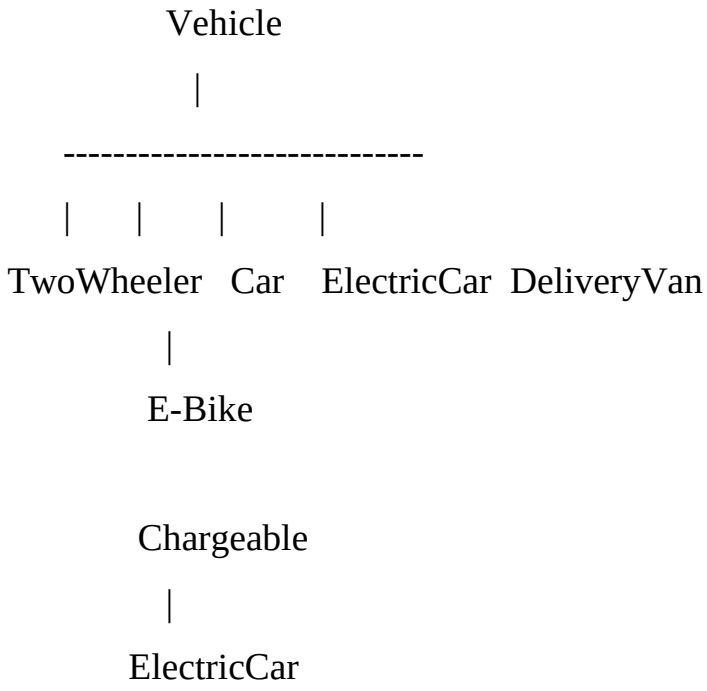
Therefore, pricing should be separated into an independent pricing component.

C. ADAPTATION OR REDESIGN

Selected Decision: MODIFY THE ORIGINAL DESIGN.

The original Vehicle hierarchy is retained, but the pricing logic and pass management are improved.

Revised Class Structure



```
          ParkingPass
              |
     ---------------------
     |                   |
  MonthlyPass         OneDayPass

          PricingRule
              |
       ParkingService
```

1. E-Bike Class

```
class EBike extends Vehicle {

    EBike(String regNo) {
        super(regNo);
    }

    @Override
    protected double ratePerHour() {
        return 15;
    }

    @Override
    public double calculateFee(int minutes) {
        if (minutes <= 0) return 0;
        return 20;
```

```
    }
}
```

The e-bike uses a flat parking fee of ₹20 per parking session.

2. DeliveryVan Class

```
import java.time.LocalTime;

class DeliveryVan extends Vehicle {

    DeliveryVan(String regNo) {
        super(regNo);
    }

    @Override
    protected double ratePerHour() {
        return 60;
    }

    public double calculateFee(
            int minutes, LocalTime entryTime) {

        int hours = (minutes + 59) / 60;

        LocalTime start =
            LocalTime.of(9, 0);
```

```
        LocalTime end =
            LocalTime.of(17, 0);

        boolean daytime =
            !entryTime.isBefore(start) &&
            entryTime.isBefore(end);

        double rate = daytime ? 60 : 30;

        return hours * rate;
    }

    @Override
    public double calculateFee(int minutes) {
        return calculateFee(
            minutes, LocalTime.MIDNIGHT
        );
    }
}
```

The car rate is ₹30 per hour. The delivery van rate is doubled to ₹60 per hour during the specified time period.

For a production system, the pricing calculation should use the actual time intervals overlapping the parking duration, especially when a vehicle crosses the 9 AM or 5 PM boundary.

3. ParkingPass Interface

```
interface ParkingPass {

    boolean isValid();

    double discount();
}
```

4. MonthlyPass Class

```
class MonthlyPass implements ParkingPass {

    public boolean isValid() {
        return true;
    }

    public double discount() {
        return 100;
    }
}
```

5. OneDayPass Class

```
class OneDayPass implements ParkingPass {

    public boolean isValid() {
        return true;
    }
```

```
    public double discount() {
        return 100;
    }
}
```

These are simplified examples. A real implementation must validate expiry dates and associate each pass with its registered user.

6. Revised Parking Facility

```
class RevisedParkingFacility {

    private final String[][] slots;

    RevisedParkingFacility(int[] slotCounts) {

        slots = new String[slotCounts.length][];

        for (int i = 0; i < slotCounts.length; i++) {
            slots[i] = new String[slotCounts[i]];
        }
    }

    public int[] allocate(String vehicleNo) {

        for (int f = 0; f < slots.length; f++) {

            for (int s = 0; s < slots[f].length; s++) {
```

```
            if (slots[f][s] == null) {

                slots[f][s] = vehicleNo;

                return new int[]{f, s};
            }
        }
    }

    return null;
  }
}
```

D. EVIDENCE-BASED DEFENCE

The modified design is better than completely rewriting the original system.

Reasons:

1. Existing Vehicle and Car classes can be retained.
2. New vehicle types can be introduced through subclasses.
3. Pricing logic can be maintained separately.
4. The jagged array supports different floor capacities.
5. Pass management can be extended independently.
6. Existing tested methods can remain unchanged wherever their behaviour is unaffected.

New Test Cases

Test	Input	Expected Result
TC01	E-bike enters	Parking slot allocated
TC02	E-bike exits	Flat fee ₹20
TC03	Delivery van at 10 AM	₹60 per hour
TC04	Delivery van at 7 PM	₹30 per hour
TC05	Valid one-day visitor pass	Free parking
TC06	Five floors with different capacities	Successful initialization

Complexity Analysis

- Parking slot allocation: O(S), where S is the total number of slots.
- Fee calculation: O(1) for the simplified rate-based calculation.
- Pass validation: O(1) when the required pass information is directly available.
- Parking facility storage: O(S).

Final Conclusion

After evaluating the changed requirements, modifying the original design is the most appropriate decision.

The modified design supports additional vehicle categories, new pricing conditions, one-day passes and different floor capacities without requiring a complete rewrite.

The use of inheritance, abstraction, interfaces, encapsulation and polymorphism makes the system flexible, reusable and maintainable.

OVERALL CONCLUSION

The Campus Smart-Parking System demonstrates how Java programming and object-oriented concepts can be applied to solve a real-world problem.

The assignment covers data types, arrays, control statements, classes, objects, constructors, static members, inheritance, method overriding, polymorphism, abstraction, interfaces, packages and encapsulation.

The system automates parking allocation, ticket generation, parking fee calculation and vehicle exit management.

The final modified design can accommodate new vehicle categories and changing pricing rules. Therefore, object-oriented programming provides a suitable foundation for developing an extensible Campus Smart-Parking System.